

第 26 讲

NumPy 手写完整 Backpropagation + Training Loop

从“会算梯度”到“让网络真的学起来”

《面向 LLM / Agent Systems 科研的深度学习零基础课》

【本讲定位】前 25 讲我们已经把 Forward、Loss、Derivative、Partial Derivative、Gradient、Chain Rule、Computational Graph 和两层 MLP Backpropagation 分别拆开。本讲不再增加新的数学负担，而是第一次把它们放进一个循环：Forward $\rightarrow$ Loss $\rightarrow$ Backward $\rightarrow$ Update $\rightarrow$ 再 Forward。当这个闭环重复执行，网络才真正进入“训练 (training)”。

学习目标

学完本讲，你应该能够：

1. 不用 PyTorch，自己写出两层 MLP 的完整训练循环；
2. 解释一个 training step 内部到底发生了什么；
3. 看懂并检查 $dW = X^T dY$ 、 $db = \sum dY$ 、 $dX = dY W^T$ 在代码中的对应关系；
4. 理解为什么参数更新必须沿负梯度方向；
5. 区分 step、training loop、loss curve、gradient norm；
6. 用 numerical gradient check 判断手写 Backward 是否可能写错；
7. 理解 learning rate 太小、合适、太大时训练会出现什么现象；
8. 明确“training loss 下降”不等于“模型泛化好”。

1 【原理】训练到底是什么？

前面我们一直在做“单次计算”。例如给定参数 W_1, b_1, W_2, b_2 ：

$$X \rightarrow Z_1 = XW_1 + b_1 \rightarrow A_1 = \text{ReLU}(Z_1) \rightarrow \hat{Y} = A_1W_2 + b_2$$

再由目标 Y 算：

$$L = \frac{1}{N} \sum_{i=1}^N (\hat{y}_i - y_i)^2$$

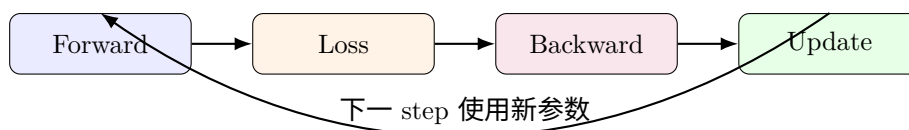
Backward 能告诉我们：

$$\nabla L = [dW_1, db_1, dW_2, db_2]$$

但是到这里，网络参数**还没有变**。训练真正发生在：

$$\theta \leftarrow \theta - \eta \nabla_{\theta} L$$

这里 θ 代表所有参数， η 是 learning rate。



训练的本质不是“运行 model 很多次”，而是：每一次 Forward 之后，都依据 Loss 计算梯度，再用梯度修改参数。如果参数从未更新，那么重复 Forward 1000 次也不是训练。

2 【通俗解释】像“下山”一样重复纠错

可以把参数想成你站在 Loss Landscape 上的位置：

- Forward：看看当前位置会得到什么预测；
- Loss：测量当前位置有多高；
- Backward：计算当前坡度；
- Update：沿负梯度方向走一小步；
- 下一 step：站在新位置重新测量。

一次 update 只能利用**当前位置附近**的局部信息，所以训练必须不断重新计算。

【常见误解】“既然已经知道梯度方向，为什么不一直沿这个方向走？”因为参数移动以后，Loss Landscape 的局部坡度会改变。上一位置的梯度只描述上一位置附近。每走一步，原则上都要重新 Forward / Backward。

3 【具体数字例子】一个最小 Training Step

先用一维参数理解，不急看矩阵。假设：

$$L(w) = (2w - 6)^2$$

当前 $w = 2$ ：

$$L(2) = (4 - 6)^2 = 4$$

导数：

$$\frac{dL}{dw} = 4(2w - 6)$$

因此：

$$\left. \frac{dL}{dw} \right|_{w=2} = -8$$

取 learning rate:

$$\eta = 0.05$$

参数更新:

$$w_{new} = 2 - 0.05 \times (-8) = 2.4$$

重新计算:

$$L(2.4) = (4.8 - 6)^2 = 1.44$$

Loss 从 4 降到 1.44。

注意：不是“梯度本身让 Loss 下降”。真正执行的是 $w \leftarrow w - \eta dL/dw$ 。梯度为负时，减去负数会让 w 增大；梯度为正时，参数会减小。

4 【图解】本讲真正训练的网络

我们用一个非常小的回归任务：

$$y = 2x + 1$$

输入 20 个 $x \in [-1, 1]$ ，网络结构为：

$$1 \rightarrow 8 \rightarrow 1$$

也就是：

$$X : (20, 1)$$

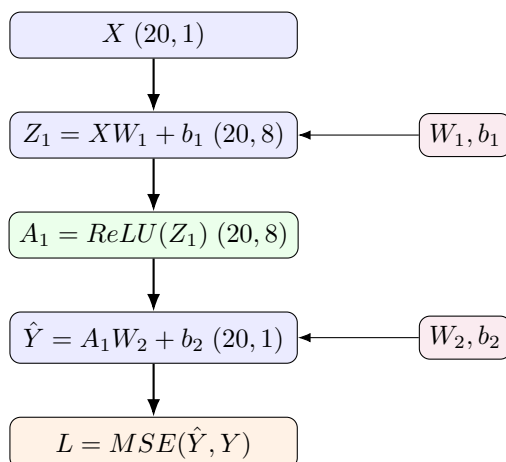
$$W_1 : (1, 8), \quad b_1 : (1, 8)$$

$$Z_1 = XW_1 + b_1 : (20, 8)$$

$$A_1 = \text{ReLU}(Z_1) : (20, 8)$$

$$W_2 : (8, 1), \quad b_2 : (1, 1)$$

$$\hat{Y} = A_1W_2 + b_2 : (20, 1)$$



5 【必要公式】Forward、Loss、Backward、Update 全部放在一起

1. Forward

$$Z_1 = XW_1 + b_1$$

$$A_1 = ReLU(Z_1)$$

$$\hat{Y} = A_1W_2 + b_2$$

2. MSE

$$L = \frac{1}{N} \sum_{i=1}^N (\hat{y}_i - y_i)^2$$

3. 输出梯度

$$\frac{\partial L}{\partial \hat{Y}} = \frac{2}{N} (\hat{Y} - Y)$$

4. 第二层 Linear Backward

$$dW_2 = A_1^T d\hat{Y}$$

$$db_2 = \sum_{i=1}^N d\hat{Y}_i$$

$$dA_1 = d\hat{Y}W_2^T$$

5. ReLU Backward

$$dZ_1 = dA_1 \odot \mathbf{1}(Z_1 > 0)$$

6. 第一层 Linear Backward

$$dW_1 = X^T dZ_1$$

$$db_1 = \sum_{i=1}^N dZ_{1,i}$$

7. Gradient Descent Update

$$W_1 \leftarrow W_1 - \eta dW_1$$

$$b_1 \leftarrow b_1 - \eta db_1$$

$$W_2 \leftarrow W_2 - \eta dW_2$$

$$b_2 \leftarrow b_2 - \eta db_2$$

6 【公式逐项拆解】为什么 Batch MSE 要除以 N ?

如果：

$$L = \frac{1}{N} \sum_i (\hat{y}_i - y_i)^2$$

那么：

$$\frac{\partial L}{\partial \hat{y}_i} = \frac{2}{N} (\hat{y}_i - y_i)$$

这里的 $1/N$ 很重要。因为我们定义的是**平均平方误差**，而不是总平方误差。

如果你忘了除以 N ，梯度会整体放大约 N 倍。这不一定让方向错误，但会改变梯度尺度，从而改变“同一个 learning rate 实际走多远”。

【科研阅读提示】以后看论文或代码，不能只看“用了 MSE / Cross Entropy”。还要看 reduction 是 mean 还是 sum、是否按 token / sample / batch 做了额外归一化，因为这些都会影响梯度尺度。

7 一个 Training Loop 的代码骨架

```
for step in range(steps):
    pred, cache = forward(X, W1, b1, W2, b2)    # 1. Forward
    loss = mse_loss(pred, Y)                    # 2. Loss
    grads = backward(pred, Y, cache)            # 3. Backward
    W1 = W1 - lr * dW1                          # 4. Update
    b1 = b1 - lr * db1
    W2 = W2 - lr * dW2
    b2 = b2 - lr * db2
```

这就是最小训练闭环。以后 PyTorch 只是把其中很多步骤封装起来。

8 【代码实验】主实验结果

固定：

- seed = 42;

- learning rate = 0.03;
- steps = 2000;
- 网络结构 $1 \rightarrow 8 \rightarrow 1$ 。

实际运行结果：

Step	MSE Loss	Gradient Norm
0	1.376293897	2.636107121
10	0.201688394	1.162390070
50	0.016090193	0.094922050
100	0.008488307	0.051503567
500	0.001005813	0.012892456
1000	0.000169043	0.003213296
1999	0.000065037	0.001765084

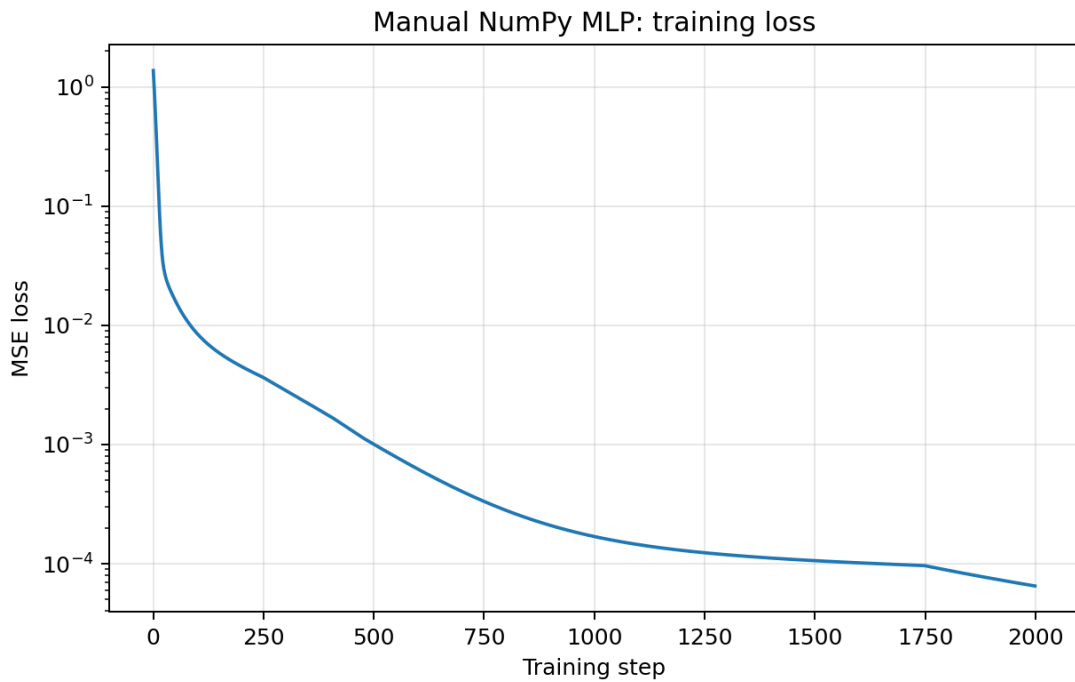


图 1: 手写 NumPy MLP 的训练 Loss。纵轴采用对数尺度，可以看到 Loss 跨多个数量级持续下降。

9 训练前和训练后，到底发生了什么？

前 5 个训练目标是：

$[-1.000, -0.789, -0.579, -0.368, -0.158]$

训练前预测：

$[0.297, 0.264, 0.231, 0.199, 0.166]$

训练后预测：

$[-0.997, -0.788, -0.578, -0.369, -0.159]$

最终：

$$MAE \approx 0.00499$$

最大绝对误差约：

$$0.02318$$

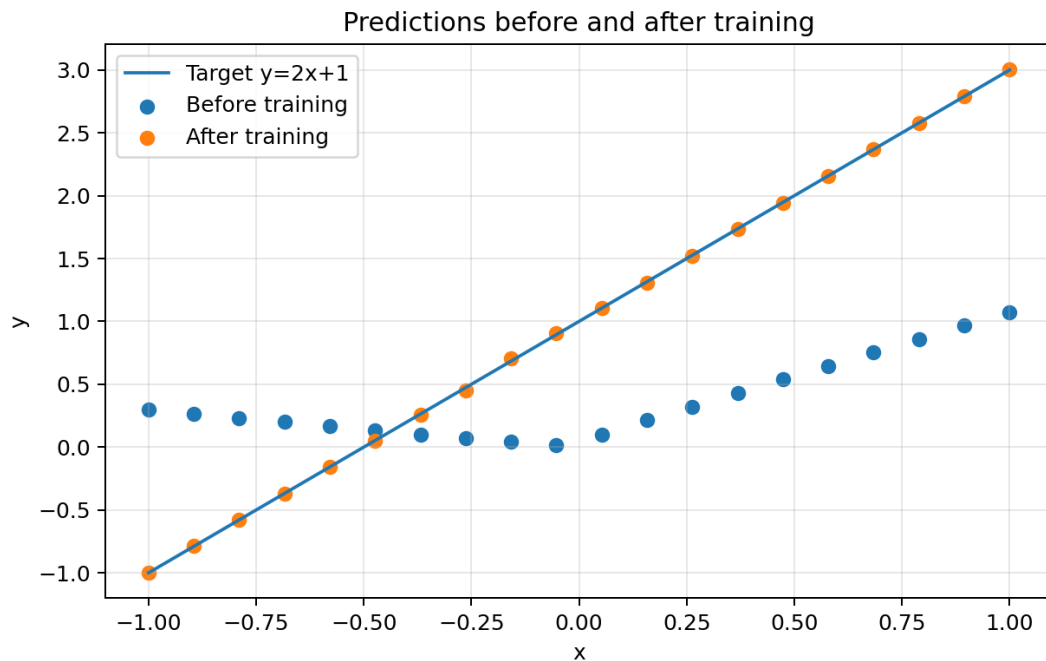


图 2: 同一个网络在训练前后的预测变化。

训练真正改变的是参数 W_1, b_1, W_2, b_2 。结构没有变化，代码结构没有变化，输入 Shape 也没有变化；变化的是参数中的数字，因此相同的输入最终产生了不同的预测。

10 Gradient Norm 是什么？

把所有参数梯度拼在一起，可以定义一个整体大小：

$$\|g\|_2 = \sqrt{\sum_k g_k^2}$$

本实验中 Gradient Norm 从约 2.636 逐渐下降到约 0.00177。

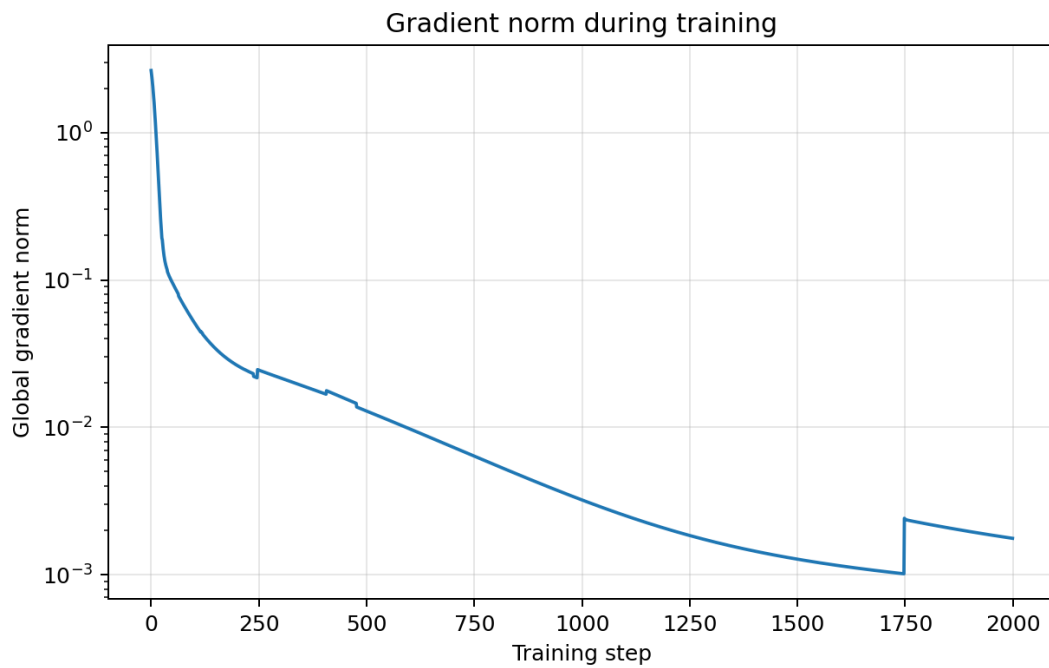


图 3: 训练过程中整体 Gradient Norm 的变化。

Gradient norm 下降通常说明模型接近某个较平缓区域，但不能机械理解为“模型一定已经学好了”。第 21 讲已经看到：高 Loss 的 plateau 也可能出现小 gradient。必须结合 Loss、Validation、任务指标一起判断。

11 Learning Rate: 同样的梯度，走多远？

为了只研究 learning rate，本实验固定：

- 相同数据；
- 相同网络结构；
- 相同 seed；
- 相同初始参数。

只改变 η 。

Learning Rate	Steps	Initial Loss	Final Loss
0.003	500	1.376294	0.016322
0.03	500	1.376294	0.001011
1.0	80	1.376294	10.717153

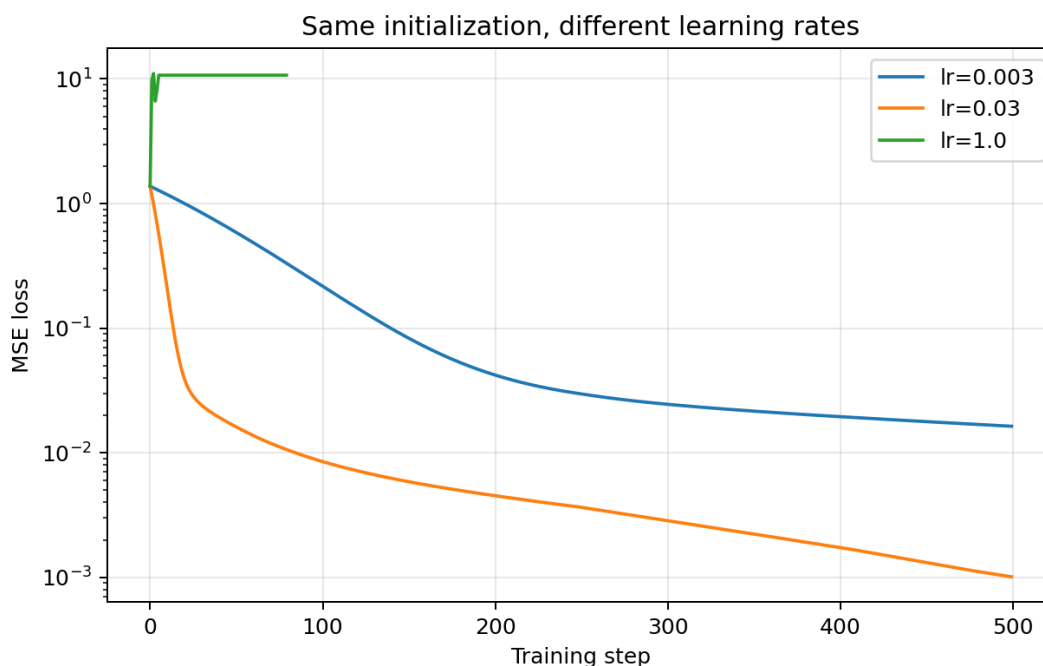


图 4: 相同初始化下, 只改变 learning rate 的训练结果。

结论不是 “0.03 永远最好”, 而是:

- 太小: 方向可能没问题, 但每一步走得太短;
- 合适: Loss 能稳定、较快下降;
- 太大: 局部方向没错, 但一步跨得太远, 可能震荡甚至发散。

12 Gradient Check: 为什么科研代码里值得做?

对于手写 Backward, 一个非常实用的验证方法是:

$$\frac{\partial L}{\partial \theta} \approx \frac{L(\theta + \epsilon) - L(\theta - \epsilon)}{2\epsilon}$$

本实验分别检查 W_1, b_1, W_2, b_2 中各一个元素:

参数	解析梯度	数值梯度	绝对误差
$W_1[0, 0]$	0.007174907	0.007174907	2.5×10^{-12}
$b_1[0, 0]$	0.013030021	0.013030021	1.5×10^{-11}
$W_2[0, 0]$	-0.445614491	-0.445614491	7.1×10^{-13}
$b_2[0, 0]$	-1.258798161	-1.258798161	5.6×10^{-12}

【科研习惯】 当你自己实现新的 Loss、特殊层、手写 backward 或复杂数学模块时, 如果可以用小输入做 finite-difference gradient check, 它往往能在正式跑实验前发现符号、转置、归一化和 Shape 方面的错误。

13 Training Step、Epoch、Batch：现在先建立接口

本讲使用整个 20 样本数据一次计算，所以每个 step 都看到了全部数据。以后进入 mini-batch 后：

- **Batch**：一次 Forward / Backward 使用的一小批样本；
- **Step**：完成一次参数更新；
- **Epoch**：整个训练集大致被遍历一遍。

第 35 讲会系统处理它们。现在只要记住：**参数每 update 一次，通常就记作一个 optimizer step。**

14 【常见误区】

1. **Loss 下降 = 泛化好**。错误。本讲只有训练数据，没有独立 validation/test。
2. **梯度算对 = 一定训练稳定**。错误。Learning rate、初始化、数值尺度都会影响训练。
3. **参数更新用 $\theta + \eta g$** 。普通 Gradient Descent 是 $\theta - \eta g$ 。
4. **Backward 只算 Weight 梯度**。Bias 同样是参数；而且还需要把梯度传回前层。
5. **Loss 每一步都必须严格下降**。在 mini-batch SGD 中不一定；不同 batch 会带来噪声。当前 full-batch 小例子才比较平滑。
6. **Gradient norm 越小越好**。不能单独判断；可能接近好解，也可能处在 plateau 或出现 vanishing gradient。
7. **把训练步数无限增大就一定更好**。后面会看到 overfitting、计算成本和早停等问题。

15 【与以前 LLM 课程的关系】现在重新理解 LLM Pretraining

你以前在 LLM 课程里见过：

Token IDs → Embedding → Transformer → Logits → Cross Entropy → Backprop → Optimizer

现在应该把它还原成和本讲同一个闭环：



区别只是 LLM 的参数从几十个、几百个变成几十亿甚至更多，Loss 从简单 MSE 变成 token-level Cross Entropy，Optimizer 从裸 Gradient Descent 变成 AdamW 等更成熟方法。

16 【与 Transformer / Agent Systems 的关系】

Transformer

本讲训练的是一个很小的 MLP，但 Transformer Block 中的 Q/K/V projection、Attention 输出投影、FFN/MLP、LayerNorm 参数等最终都必须接收梯度并被更新。底层仍然是：

Forward → Loss → Backward → Update

Agent Systems

很多 Agent Systems 研究不需要你自己训练基础模型，但你仍需要判断：

- 一个模型是 frozen 还是 fine-tuned?
- 哪些参数参与更新?
- Loss 是什么?
- 训练多少 steps / epochs?
- learning rate 是否合理?
- 对照实验是否使用相同初始化和训练预算?

因此“训练闭环”的理解仍然是科研最低基础。

17 【科研边界】本实验证明了什么，没有证明什么？

本实验支持的结论：手写 Forward / Backward 与数值梯度一致；在当前小数据和超参数下，沿负梯度重复更新能显著降低训练 MSE；learning rate 对优化稳定性有明显影响。

本实验没有证明：网络对未见数据泛化好；这个 MLP 是最佳结构；0.03 是普遍最佳 learning rate；Loss 更低一定意味着实际任务更好；训练 2000 steps 一定是合理预算。

18 【练习题】

1. 用一句话区分 Forward、Backward、Update。
2. 如果 $dL/dw = 5$ ， $lr = 0.01$ ，当前 $w = 2$ ，更新后 w 是多少？
3. 如果 Batch MSE 使用 mean reduction，为什么 $d\hat{Y}$ 中会出现 $1/N$ ？
4. 对 $X : (32, 10)$ 、 $W : (10, 20)$ ，Forward 输出 Shape 是什么？如果输出梯度 $dY : (32, 20)$ ， dW 的 Shape 是什么？
5. 为什么 gradient check 更适合用很小的模型和很小的数据做，而不是在大模型上逐参数做？
6. learning rate 太大会出现什么现象？“梯度方向正确”为什么仍可能训练失败？
7. Training Loss 从 1.0 降到 0.001，能否推出 Test Loss 也会下降？为什么？

19 【答案】

1. Forward 用当前参数算预测；Backward 计算 Loss 对参数的梯度；Update 用梯度修改参数。
2. $w_{new} = 2 - 0.01 \times 5 = 1.95$ 。

3. 因为 Loss 定义为 N 个样本平方误差的平均；对单个预测求导自然继承 $1/N$ 。
4. Forward: $(32, 20)$; $dW = X^T dY$, 所以 $(10, 32) @ (32, 20) = (10, 20)$ 。
5. 数值梯度每检查一个参数元素都要额外做 Forward, 成本很高; 小模型足以检查公式/实现是否正确。
6. 一步可能跨过低 Loss 区域, 造成震荡或发散。Gradient 只提供当前位置的局部方向, 不保证任意大步长都下降。
7. 不能。训练集与测试集不同, 还可能过拟合; 需要独立 Validation/Test 才能讨论泛化。

本讲最低闭环验收

如果你能不看答案解析下面这段伪代码每一行的含义, 本讲就达标:

```
for step in range(steps):  
    pred, cache = forward(X, params)  
    loss = mse(pred, Y)  
    grads = backward(pred, Y, cache)  
    params = params - lr * grads
```

下一讲将正式进入 PyTorch: 我们会把今天自己写的 Tensor、Linear、参数、Forward 和 Backward 分别替换成 torch.Tensor、nn.Linear 和 Autograd, 并逐项对照 “PyTorch 到底替我们做了什么”。